

Web Development

Labo 19

Jari Rhellam

Timers

In JavaScript zijn er twee manieren om met timers te werken:

`setInterval` — voert code herhaaldelijk uit om de zoveel milliseconden. Geeft een ID terug om de timer later te stoppen met `clearInterval`.

`setTimeout` — voert code eenmalig uit na een wachttijd in milliseconden. Geeft een ID terug om de timer te annuleren met `clearTimeout`.

Timings zijn niet perfect: de browser is single-threaded. Als er andere code bezig is, kan een timer iets later afvuren dan gepland.

Demo timers — Demo 01: `setInterval`

`setInterval` roept een functie herhaaldelijk op. Het teruggegeven ID sla je op zodat je de timer later kan stoppen met `clearInterval`.

```
let timerId = 0;

const initialize = () => {
  document.getElementById("btnStop").addEventListener("click", stopTimer);
  timerId = setInterval(timerTick, 1000);
};

const timerTick = () => {
  document.getElementById("output").innerHTML += " tick";
};

const stopTimer = () => {
  clearInterval(timerId);
};

window.addEventListener("load", initialize);
```

Observatie: elke seconde verschijnt 'tick' in de output. De knop stopt de timer via `clearInterval` met het opgeslagen ID. Zonder dat ID zou de timer niet te stoppen zijn.

Demo timers — Demo 02: setTimeout eenmalig

setTimeout voert een functie slechts eenmaal uit na de opgegeven wachttijd. Wil je het herhalen, moet je opnieuw setTimeout aanroepen.

```
const initialize = () => {
  document.getElementById("btnOpnieuw").addEventListener("click", herstartTimer);
  setTimeout(timerTick, 1000);
};

const timerTick = () => {
  document.getElementById("output").innerHTML += " tick";
};

const herstartTimer = () => {
  setTimeout(timerTick, 1000);
};

window.addEventListener("load", initialize);
```

Observatie: slechts één 'tick' na 1 seconde. De knop plant opnieuw een eenmalige timeout. Zonder de knop verschijnt er niets meer na de eerste tick.

Demo timers — Demo 03: setTimeout herhaald

Door in de callback telkens opnieuw setTimeout aan te roepen, simuleer je setInterval. Voordeel: de wachttijd begint pas NA de uitvoering van de vorige callback.

```
let timerId = 0;

const initialize = () => {
  document.getElementById("btnStop").addEventListener("click", stopTimer);
  timerId = setTimeout(timerTick, 1000);
};

const timerTick = () => {
  document.getElementById("output").innerHTML += " tick";
  timerId = setTimeout(timerTick, 1000); // plant zichzelf opnieuw
};

const stopTimer = () => {
  clearTimeout(timerId);
};

window.addEventListener("load", initialize);
```

Observatie: zelfde visueel resultaat als Demo 01, maar de timer herplant zichzelf.
Stoppen met `clearTimeout` en het laatste opgeslagen `timerId`.

Duurtijd bepalen

Er zijn twee manieren om de duurtijd van code te meten. `console.time()` is eenvoudiger en accurater:

```
// Methode 1: Date.now()
let start = Date.now();
// ... code ...
console.log(Date.now() - start + " ms");

// Methode 2: console.time() — aanbevolen
console.time("mijnCode");
// ... code ...
console.timeEnd("mijnCode"); // logt automatisch de duurtijd
```

Willekeurige getallen

`Math.random()` geeft een kommagetal in $[0, 1[$. Door te vermenigvuldigen en op te tellen kan je elk gewenst bereik bekomen:

```
Math.random()      // kommagetal in [0, 1[
Math.random() * 15  // kommagetal in [0, 15[
10 + Math.random() * 15 // kommagetal in [10, 25[

// Afronden naar geheel getal:
Math.floor(7.67) // -> 7 (altijd naar beneden)
Math.ceil(7.34)  // -> 8 (altijd naar boven)
Math.round(7.5)  // -> 8 (wiskundig)

// Willekeurig geheel getal in [0, 4]:
Math.floor(Math.random() * 5)
```

Let op met `Math.round` bij negatieve getallen: `Math.round(-7.5)` geeft -7, niet -8. Gebruik `Math.floor` als je altijd naar beneden wil afronden.

Globale variabelen

Globale variabelen zijn in de browser properties van het window object. Risico op naamconflicten is groot. Beste aanpak: alles bundelen in één object:

```
let global = {
  IMAGE_COUNT: 5,    // constante
  MOVE_DELAY: 3000,  // constante
  score: 0,          // spelerstoestand
  timeoutId: 0       // timer-id zodat je kan annuleren
};

// gebruik:
global.score++;
clearTimeout(global.timeoutId);
```

Voordeel: bij het debuggen open je één Watch voor 'global' en zie je meteen alle waarden. Een globale variabele gaat verloren als de pagina herlaadt of de browser navigeert.

Opdracht: Hit an Object

Een interactief spel waarbij elke seconde een figuur op een willekeurige positie verschijnt. Klikken op een gewone figuur geeft een punt. Klikken op de bom (0.png) eindigt het spel. setTimeout plant telkens een nieuwe move() na MOVE_DELAY ms.

HTML (index.html):

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <meta charset="UTF-8">
  <link rel="stylesheet" href="styles/style.css">
  <title>Hit an Object</title>
</head>
<body>
  <div id="playField">
    <button id="btnStart">START</button>
    Aantal hits <span class="score">0</span>
    
  </div>
  <script src="scripts/code.js"></script>
</body>
</html>
```

JavaScript (scripts/code.js):

```
let global = {
  IMAGE_COUNT: 5,
  IMAGE_SIZE: 48,
  IMAGE_PATH_PREFIX: "images/",
  IMAGE_PATH_SUFFIX: ".png",
```

```

    MOVE_DELAY: 3000,
    score: 0,
    timeoutId: 0
  };

  const setup = () => {
    document.getElementById("btnStart").addEventListener("click", startGame);
  };

  const startGame = () => {
    document.getElementById("btnStart").style.display = "none";
    let target = document.getElementById("target");
    target.addEventListener("click", klik);
    move();
  };

  const klik = (ev) => {
    if (ev.target.className.indexOf("bom") !== -1) {
      gameOver();
    } else {
      hit();
    }
  };

  const move = () => {
    let target = document.getElementById("target");
    let speelScherf = document.getElementById("playField");
    let maxLeft = speelScherf.clientWidth - global.IMAGE_SIZE;
    let maxTop = speelScherf.clientHeight - global.IMAGE_SIZE;
    let nummer = Math.floor(Math.random() * global.IMAGE_COUNT);
    target.className = (nummer === 0) ? "bom" : "";
    target.setAttribute("src", global.IMAGE_PATH_PREFIX + nummer +
global.IMAGE_PATH_SUFFIX);
    target.style.left = Math.floor(Math.random() * maxLeft) + "px";
    target.style.top = Math.floor(Math.random() * maxTop) + "px";
    global.timeoutId = setTimeout(move, global.MOVE_DELAY);
  };

  const gameOver = () => {
    clearTimeout(global.timeoutId);
    alert("GAME OVER! Score: " + global.score);
  };

  const hit = () => {
    clearTimeout(global.timeoutId);
    global.score++;
    let scoreSpans = document.getElementsByClassName("score");

```

```
for (let i = 0; i < scoreSpans.length; i++) {  
    scoreSpans[i].innerText = global.score;  
}  
move();  
};  
  
window.addEventListener("load", setup);
```

Hoe het werkt: de class 'bom' op het img-element bepaalt of het een bom is — zo hoeft je de src niet te ontleden in de click handler. Bij een hit annuleer je de lopende timeout met clearTimeout en roep je meteen move() opnieuw aan. Bij game over stop je de timer definitief.

Opdracht: Matching Game

Een memory-spel met 12 afbeeldingen (elk 2 keer = 24 kaarten). Kaarten worden willekeurig geschud. Bij twee gelijke kaarten worden ze na 1 seconde verwijderd, bij twee verschillende kaarten worden ze teruggedraaid. Het speelveld is geblokkeerd tijdens de wachttijd.

Belangrijke aandachtspunten:

LIVE collections: `getElementsByClassName` geeft een live lijst. Bij verwijderen of klassewijziging in een for-lus sla je elementen over. Gebruik `while (collection.length > 0)`.

`data-imageSource`: de voorkant-afbeelding wordt opgeslagen als data-attribuut op de kaart zodat je de src niet opnieuw hoeft te berekenen bij omdraaien.

Blokking: `playField.className = 'geblokkeerd'` voorkomt klikken tijdens de wachttijd na twee omgedraaide kaarten.

HTML (index.html):

```
<!DOCTYPE html>  
<html lang="nl">  
<head>  
    <meta charset="UTF-8">  
    <title>Matching Game</title>  
    <link rel="stylesheet" href="css/styles.css">  
</head>  
<body>  
    <div id="playField"></div>  
    <audio id="draai" preload="auto">  
        <source src="sound/draai.mp3" type="audio/mpeg">  
        <source src="sound/draai.wav" type="audio/wav">  
    </audio>
```

```

<audio id="goed" preload="auto">
  <source src="sound/goed.mp3" type="audio/mpeg">
  <source src="sound/goed.wav" type="audio/wav">
</audio>
<audio id="fout" preload="auto">
  <source src="sound/fout.mp3" type="audio/mpeg">
  <source src="sound/fout.wav" type="audio/wav">
</audio>
<script src="script/code.js"></script>
</body>
</html>

```

JavaScript (script/code.js):

```

let global = {
  AANTAL_AFBEELDINGEN: 12,
  AANTAL_KAARTEN_PER_AFBEELDING: 2,
  AANTAL_KAARTEN_HORIZONTAAL: 6,
  PREFIX_KAART_PATH: "image/kaart",
  SUFFIX_KAART_PATH: ".png",
  ACHTERKANT_PATH: "image/achterkant.png",
};

const shuffle = (array) => {
  array.sort(() => Math.random() - 0.5);
};

const toonGoed = () => {
  document.getElementById("goed").play();
  let kaarten = document.getElementsByClassName("voorkant");
  for (let i = 0; i < kaarten.length; i++) {
    kaarten[i].className += " goed";
  }
};

const toonFout = () => {
  document.getElementById("fout").play();
  let kaarten = document.getElementsByClassName("voorkant");
  for (let i = 0; i < kaarten.length; i++) {
    kaarten[i].className += " fout";
  }
};

const draaiKaartenMetVoorkantNaarAchterkant = () => {
  let kaarten = document.getElementsByClassName("voorkant");
  // LIVE collection: gebruik while i.p.v. for!
  while (kaarten.length > 0) {

```

```

    kaarten[0].setAttribute("src", global.ACHTERKANT_PATH);
    kaarten[0].className = "kaart achterkant";
  }
  document.getElementById("playField").className = "";
};

const verwijderKaartenMetVoorkant = () => {
  let kaarten = document.getElementsByClassName("voorkant");
  // LIVE collection: gebruik while i.p.v. for!
  while (kaarten.length > 0) {
    let kaart = kaarten[0];
    kaart.removeEventListener("click", klikOpKaart);
    kaart.parentNode.removeChild(kaart);
  }
  document.getElementById("playField").className = "";
  controleerSpelGedaan();
};

const controleerSpelGedaan = () => {
  let kaarten = document.getElementsByClassName("kaart");
  if (kaarten.length === 0) {
    let playField = document.getElementById("playField");
    let savedHeight = playField.clientHeight;
    playField.innerHTML = "";
    playField.style.height = savedHeight + "px";
    playField.className = "klaar";
  }
};

const controleerOpOvereenkomst = (kaarten) => {
  let allenGelijk = true;
  for (let i = 1; i < kaarten.length; i++) {
    if (kaarten[0].getAttribute("src") !== kaarten[i].getAttribute("src")) {
      allenGelijk = false;
      break;
    }
  }
  if (allenGelijk) {
    setTimeout(toonGoed, 500);
    setTimeout(verwijderKaartenMetVoorkant, 1000);
  } else {
    setTimeout(toonFout, 500);
    setTimeout(draaiKaartenMetVoorkantNaarAchterkant, 1000);
  }
};

const klikOpKaart = (e) => {

```



```

if (document.getElementById("playField").className === "geblokkeerd") return;
let kaart = e.target;
document.getElementById("draai").play();
kaart.className = "kaart voorkant";
kaart.setAttribute("src", kaart.getAttribute("data-imageSource"));
let kaartenMetVoorkant = document.getElementsByClassName("voorkant");
// LIVE: onze kaart zit er al in, test op ==2 niet ==1
if (kaartenMetVoorkant.length === global.AANTAL_KAARTEN_PER_AFBEELDING) {
  controleerOpOvereenkomst(kaartenMetVoorkant);
  document.getElementById("playField").className = "geblokkeerd";
}
};

const addVak = (parent, kaartNummer) => {
  let vak = document.createElement("span");
  let kaart = document.createElement("img");
  kaart.className = "kaart achterkant";
  kaart.setAttribute("src", global.ACHTERKANT_PATH);
  kaart.setAttribute("data-imageSource",
    global.PREFIX_KAART_PATH + kaartNummer + global.SUFFIX_KAART_PATH);
  kaart.addEventListener("click", klikOpKaart);
  vak.className = "vak";
  vak.appendChild(kaart);
  parent.appendChild(vak);
};

const initialize = () => {
  let playField = document.getElementById("playField");
  let aantalKaarten = global.AANTAL_AFBEELDINGEN *
global.AANTAL_KAARTEN_PER_AFBEELDING;
  let kaartNummers = [];
  for (let i = 0; i < aantalKaarten; i++) {
    kaartNummers.push(i % global.AANTAL_AFBEELDINGEN);
  }
  shuffle(kaartNummers);
  for (let i = 0; i < aantalKaarten; i++) {
    if (i % global.AANTAL_KAARTEN_HORIZONTAAL === 0 && i !== 0) {
      playField.appendChild(document.createElement("br"));
    }
    addVak(playField, kaartNummers[i]);
  }
};

window.addEventListener("load", initialize);

```

De twee setTimeout-aanroepen per uitkomst (500ms voor feedback, 1000ms voor actie) zorgen dat de gebruiker eerst de kleur ziet (groen/rood border) en dan pas de kaarten

verdwijnen of terugdraaien. `removeEventListener` voor het verwijderen van kaarten is belangrijk om geheugenlekken te vermijden.